

A Mechanically Verified Translation of $MTL_{0,\infty}$ into Timed Büchi Automata

Elie Fares, Jean-Paul Bodeveix, Hussain Al-Aqrabi, and Azar Salami



Abstract—We present a Rocq-verified semantic translation from pointwise dense-time $MTL_{0,\infty}$ to event-guarded timed Büchi automata, without location invariants and with existential initial clock valuations. The mechanized theorem covers well-formed core formulas: lower and strict upper bounds are positive, and non-strict upper bounds are nonnegative. We describe reductions for zero lower bounds and zero strict upper bounds, but do not mechanize this preprocessing. Ordinary timed operators are expressed through hatted Until and Release primitives, encoded in clocked LTL, and passed to an LTL-to-Büchi backend satisfying an explicit semantic contract. Relaxation and reset completion yield a timed Büchi automaton. The proof assigns one clock to each distinct primitive timed subformula, including overlapping activations. Soundness holds for every clock-consistent reset trace; completeness constructs a canonical trace using residual obligations. These arguments establish `MTL_to_TBA_correct_with`. The result covers strict and non-strict bounds within the stated core domain; it is an abstract translation result, not an integrated compiler or a performance comparison.

Index Terms—Metric temporal logic, timed Büchi automata, Rocq, formal verification, clocked LTL, semantic preservation.

1 INTRODUCTION

Real-time requirements combine qualitative ordering with quantitative timing. Metric Temporal Logic (MTL) expresses both, while timed Büchi automata (TBA) provide a finite-state model suitable for timed verification. A translation between them is useful only when the automaton preserves the pointwise dense-time semantics of the formula. The translation itself is subtle: a timed subformula nested under an unbounded temporal operator may be activated without bound, and several instances of the same obligation may be pending at once.

A concrete target is dense-time verification of a bounded-response property such as $\Box(req \Rightarrow \Diamond_{\leq d}ack)$. If a second request arrives before an acknowledgment, two response obligations overlap. This creates a specific design question for a property-automaton generator: can one clock per timed subformula preserve the older deadline across later activations? Table 2 gives a trace, and the Rocq theorem answers this for the abstract translation under its backend contract.

This paper gives a mechanized translation for well-formed core formulas in the one-sided-bounded fragment

$MTL_{0,\infty}$ [1]–[3]. Zero-bound source cases are reduced as described below; that preprocessing is not mechanized. Its key construction assigns a clock to each *distinct primitive timed subformula*, rather than to each syntax-tree occurrence or run-time activation. The clocked-LTL encoding constrains that shared clock through reset and preservation markers. The proof’s completeness direction chooses a canonical reset trace that retains the oldest useful reference when it subsumes newer obligations; residual predicates express what remains of an obligation after time has advanced. This establishes that repeated activations need no additional clocks.

The translation is factored through ordinary LTL. It is therefore independent of the particular LTL-to-Büchi translator, subject only to a stated correctness contract on propositional words. After the backend, relaxation removes irrelevant negative extended-atom literals, and reset completion turns the relaxed cubes into timed transitions. The resulting theorem is mechanized in Rocq 9.0.1 (formerly Coq) for all eight hatted timed primitives, including strict bounds. The proof does not cover parsing, the interface to an external LTL translator, or output printing; these are outside the theorem. The Rocq syntax stores bounds as elements of $\mathbb{R}$, so the result establishes semantic correctness for Rocq-level formulas; it does not provide a finite textual encoding or an extracted executable compiler for numeric bounds.

Our contributions are:

- a pointwise-semantics-preserving clocked-LTL encoding of the eight hatted Until and Release primitives, with ordinary timed operators derived by semantic identities;
- soundness and completeness of that encoding, including a canonical reset policy and residual obligations that justify one clock per distinct primitive timed subformula despite arbitrarily overlapping activations, without dropping older pending deadlines;
- relaxation and reset completion, with a Rocq semantic theorem for every LTL-to-Büchi backend satisfying the explicit correctness contract.

The formal development includes the named results `EncodingCorrect_proved`, `T_uses_every_clock`, and `MTL_to_TBA_correct_with`. The complete arguments for the canonical policy and its strict-bound cases appear in the supplementary material. A companion manuscript reuses this Rocq translation proof and adds

Elie Fares, Hussain Al-Aqrabi, and Azar Salami are with the Higher Colleges of Technology, Sharjah, United Arab Emirates (e-mail: efares@hct.ac.ae; haaqrabi@hct.ac.ae; asalami@hct.ac.ae).

Jean-Paul Bodeveix is with IRIT, Université de Toulouse, France (e-mail: bodeveix@irit.fr).

generic TBA optimization and UPPAAL export; those downstream results are outside this paper's scope.

For a generator implementing this translation, the assurance contract is concrete: given a well-formed core Rocq formula and a propositional Büchi backend satisfying Assumption 4.1, the verified translation yields an event-guarded TBA that preserves the formula's timed language under existential initial clock values. Within this boundary, one clock per distinct primitive timed subformula remains complete across arbitrarily many overlapping activations. The trace in Table 2 illustrates the software risk this invariant addresses. With a deadline of five time units, a request at time 0 is still pending when a second request arrives at time 4. Resetting the shared clock for that second request makes an acknowledgment at time 6 appear to have age 2, although the first request's deadline expired at time 5. The theorem rules out this reset-at-each-request behavior for the proved construction. Its canonical reset trace is a completeness witness, not a future-aware policy that generated software must execute. The result justifies this clock-sharing architecture; it establishes neither a clock-count advantage over CASAAL nor a runtime or automaton-size improvement.

To transfer this result into an executable tool, a consumer must implement the target semantics: event-guarded transitions without location invariants and existential initial clock values. Parsing, finite textual encoding of bounds, backend invocation and validation, and automaton export require separate correctness arguments. A consumer that requires zero-initialized clocks needs a separately verified adapter. Fig. 4 documents the Spot 2.16 propositional interface only: the script regenerates the propositional graph, does not establish Spot's contract, and does not generate the reset-completed TBA. The worked TBA is hand-derived, so this paper reports a verified translation component rather than a formula-to-checker execution.

Neither translation to timed automata nor clocks and reset policies indexed by timed subformulas originate here. Prior constructions already establish these ideas for their stated fragments. Our contribution is the Rocq proof of this clocked-LTL factorization and its relaxation/reset-completion pipeline, with a completeness argument for repeated overlapping activations and all eight strict and non-strict primitive cases. The result is not a claim of a new clock-sharing principle, fewer clocks than prior tools, or a comparative assurance ranking; Section 6 gives the construction-level comparison.

2 LOGIC AND SEMANTIC MODEL

2.1 Pointwise MTL_{0,∞}

Let Σ be a finite set of actions. Formulas are in negation normal form and use Boolean operators, next, untimed Until and Release, and timed Until and Release whose intervals have one endpoint at zero or infinity. We write a bound as $\sim d$, with $\sim \in \{<, \leq, >, \geq\}$. The logic also contains the hatted operators $\hat{U}_{\sim d}$ and $\hat{R}_{\sim d}$. In a hatted operator the current position is skipped by the logical interval, while its timestamp remains the timing reference.

A timed word is $w = (a_i, t_i)_{i \in \mathbb{N}}$, where $a_i \in \Sigma$, $0 \leq t_0 < t_1 < \dots$, and timestamps diverge. We write $\text{elap}_w(i, j) = t_j - t_i$. For Until, $w, i \models_{\text{MTL}} p U_{\sim d} q$ holds when some $j \geq i$

satisfies q and $t_j - t_i \sim d$, with p at each position in $[i, j)$. For hatted Until, the witness is $j > i$ and p is required on (i, j) . Release is the dual universal condition: at every position in the bounded region, either q holds or p held earlier in the corresponding interval. Hatted Release excludes the current position from that region. We use the standard pointwise semantics [4].

For precision, the four bounded clauses used below are:

$$\begin{aligned}
 w, i \models_{\text{MTL}} p U_{\sim d} q &\iff \exists j \geq i. w, j \models_{\text{MTL}} q \\
 &\quad \wedge t_j - t_i \sim d \\
 &\quad \wedge \forall k \in [i, j). w, k \models_{\text{MTL}} p, \\
 w, i \models_{\text{MTL}} p \hat{U}_{\sim d} q &\iff \exists j > i. w, j \models_{\text{MTL}} q \\
 &\quad \wedge t_j - t_i \sim d \\
 &\quad \wedge \forall k \in (i, j). w, k \models_{\text{MTL}} p, \\
 w, i \models_{\text{MTL}} p R_{\sim d} q &\iff \forall j \geq i. \\
 &\quad \left(t_j - t_i \sim d \Rightarrow \right. \\
 &\quad \left. \left(w, j \models_{\text{MTL}} q \right. \right. \\
 &\quad \left. \left. \vee \exists k \in [i, j). w, k \models_{\text{MTL}} p \right) \right), \\
 w, i \models_{\text{MTL}} p \hat{R}_{\sim d} q &\iff \forall j > i. \\
 &\quad \left(t_j - t_i \sim d \Rightarrow \right. \\
 &\quad \left(w, j \models_{\text{MTL}} q \right. \\
 &\quad \left. \vee \exists k \in (i, j). w, k \models_{\text{MTL}} p \right) \right).
 \end{aligned}$$

The index intervals refer to event positions, whereas $t_j - t_i$ is physical time. This distinction is material when no event occurs exactly at a bound. The strict increase assumption excludes simultaneous events; divergence is used in the lower-bounded Until fairness argument.

The Rocq development represents action names by natural numbers rather than a parameterized finite type, and timed words may use any natural-number name. Each formula mentions finitely many names, so its semantics and generated labels distinguish actions only by equality or inequality with those names. Equivalently, the construction operates over the finite quotient induced by those action tests; this is the finite alphabet used in the mathematical presentation.

The well-formedness condition requires $d > 0$ for every lower bound and strict upper bound, and $d \geq 0$ for a non-strict upper bound. Ordinary timed operators are definitions in the mechanized core. For upper bounds $\preceq \in \{\leq, <\}$ and lower bounds $\succeq \in \{\geq, >\}$, they are:

$$p U_{\preceq d} q \equiv q \vee (p \wedge (p \hat{U}_{\preceq d} q)), \quad (1)$$

$$p U_{\succeq d} q \equiv p \wedge (p \hat{U}_{\succeq d} q), \quad (2)$$

$$p R_{\preceq d} q \equiv q \wedge (p \vee (p \hat{R}_{\preceq d} q)), \quad (3)$$

$$p R_{\succeq d} q \equiv p \vee (p \hat{R}_{\succeq d} q). \quad (4)$$

These definitions separate the current-position case from the strictly future obligation. Their pointwise semantic compatibility, including the stated bound hypotheses, is proved in Rocq:

Theorem 2.1 (Semantic compatibility of the derived timed operators). *Under the well-formedness conditions above,*

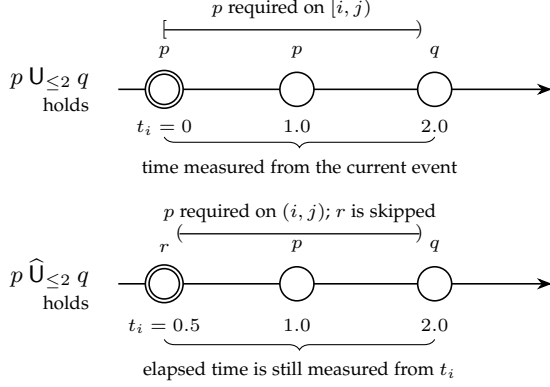


Fig. 1. Ordinary and hatted Until. The double circle is the timing reference. The hatted operator skips the current event logically but measures from its timestamp.

Eqs. (1)–(4) agree with the standard pointwise semantics for all strict and non-strict upper and lower bounds.

The identities follow by separating the current-position case from a strictly future endpoint. For upper Until, a witness at i contributes q ; every later witness requires p at i and leaves the hatted obligation. A lower bound with $d > 0$ excludes i , so p is required now and the remaining witness is hatted. The Release identities are the dual case split: an upper-bound region includes i and therefore requires q there, while a lower-bound region excludes i . Strict upper bounds also require $d > 0$, avoiding the degenerate < 0 case. The eight corresponding Rocq lemmas are identified in the result map in Supplement 3. Thus only the eight hatted timed constructors require primitive translation clauses. If a source formula has a zero lower bound, strict increase of timestamps makes every strictly future elapsed time positive. At the hatted level, $\hat{U}_{\geq 0}$ and $\hat{U}_{> 0}$ therefore reduce to $X(p \ U \ q)$, while $\hat{R}_{\geq 0}$ and $\hat{R}_{> 0}$ reduce to $X(p \ R \ q)$. For ordinary operators, the current-position case must also be simplified: the ≥ 0 cases are the corresponding untimed Until and Release operators, and the > 0 cases reduce to $p \wedge X(p \ U \ q)$ and $p \vee X(p \ R \ q)$, respectively. These reductions rely on the paper’s strictly increasing event times and must be applied before encoding a source formula as a Rocq core formula; the preprocessing itself is not mechanized. A zero strict upper bound has an empty interval: Until reduces to $\perp$ and Release to $\top$; strict upper-bound constructors in the core consequently require $d > 0$. For the non-strict upper bound zero, the hatted future operators reduce to $\hat{U}_{\leq 0} \equiv \perp$ and $\hat{R}_{\leq 0} \equiv \top$, because all strictly future events have positive elapsed time. The derived ordinary operators therefore both reduce to their right operand: $p \ U_{\leq 0} \ q \equiv q$ and $p \ R_{\leq 0} \ q \equiv q$. These reductions make the boundary explicit: the mechanized domain excludes zero lower bounds and zero strict upper bounds, but includes non-strict upper bound zero.

This distinction is useful for sporadicity: if consecutive occurrences of e must be separated by at least θ , one may write $\Box(e \Rightarrow \Box_{<\theta} \neg e)$. The hatted form retains the triggering event as the time origin; ordinary bounded temporal operators do not express this particular reference shift directly. Related pointwise MTL constructions and the CASAAL

translation are discussed in prior work [5]–[8].

2.2 Timed Büchi Automata

The verified target is an event-guarded TBA $\mathcal{A} = (L, \ell_0, X, E, Acc)$. Here L is a finite set of locations, ℓ_0 the initial location, X a finite set of clocks, $E \subseteq L \times \text{Lab} \times \text{Guard} \times 2^X \times L$ is the set of action-labelled transitions with guards and reset sets, and $Acc \subseteq L$ the Büchi set. A run is a sequence $(\ell_i, v_i)_{i \geq 0}$ starting at ℓ_0 ; it follows one transition per event and visits Acc infinitely often. If the selected edge at event i is $(\ell_i, \alpha_i, g_i, Z_i, \ell_{i+1})$, then the event action must satisfy α_i and the valuation v_i must satisfy every guard in g_i . There are no location invariants in this model: timing constraints are checked at the events of the timed word. This is the TBA record represented in Rocq; the theorem does not cover a richer invariant-bearing automaton model without an additional encoding.

Let v_i be the clock valuation tested at event i and $\Delta_i = t_{i+1} - t_i$. A transition at event i tests v_i before its reset decision; the next valuation is

$$v_{i+1}(x) = \begin{cases} \Delta_i, & x \in Z_i, \\ v_i(x) + \Delta_i, & \text{otherwise.} \end{cases}$$

The initial nonnegative valuation v_0 is existentially quantified, as in the Rocq clock-consistent-extension semantics. In the example below, the first transition resets the deadline clock at the origin event, and every outgoing transition from the waiting location checks the upper deadline. The standard zero-initialization convention is not assumed by the theorem.

3 CLOCKED-LTL TRANSLATION

3.1 Shared clock identities

The translation is structural. It maps Boolean and untimed temporal operators homomorphically and maps each primitive timed subformula to an LTL formula over an extended alphabet. For each action a , the extended alphabet contains distinct action atoms $LAct(a)$ and $LNAct(a)$, as well as clock-related atoms: clock comparisons $x \sim d$, a reset atom $rst(x)$, and a preservation atom $unch(x)$. Under timed-word interpretation, the action atoms mean that the current action is a or is different from a , respectively. The atom $LNAct(a)$ and the negated literal of $LAct(a)$ have the same timed-word interpretation, but remain distinct propositions to the unconstrained LTL backend. The reset and preservation atoms describe the clock update at the current event. Clock values are read before the event’s reset, matching the guard semantics of a timed automaton.

For formula f , let $Clock(f)$ be its finite set of distinct primitive hatted timed subformulas. The clock associated with $F \in Clock(f)$ is $x = F$ itself. Thus two identical timed subformulas at different syntax-tree paths denote the same logical clock. Syntactically different subformulas remain distinct in this construction. The main result proves that the clock also serves all dynamic activations of that subformula. The clock count is therefore bounded by the number of distinct primitive timed subformulas, which is at most the number of timed occurrences in f . If structural subformulas are represented as shared nodes, the translation can share

their LTL encodings as well; this does not remove the usual worst-case growth of an LTL-to-Büchi construction.

This sharing is semantic rather than an automaton-state minimization. The clock key is the normalized primitive formula, including its comparison and constant, so identical obligations have one clock identity before the propositional automaton is built. The reset trace is then chosen for that identity as a whole. In particular, the proof cannot justify the clock bound by assigning a fresh clock to each activation and subsequently identifying those clocks: it establishes directly that a single trace of reset decisions for the formula is sufficient.

3.2 Eight primitive clauses

Let F be a primitive hatted formula at path π , with operands p, q . Write $x = \text{clock_of}(F)$, $P = \mathcal{T}_{\pi 0}(p)$, $Q = \mathcal{T}_{\pi 1}(q)$, and define $\alpha W \beta = (\alpha U \beta) \vee G\alpha$. For upper comparisons $\lesssim \in \{\leq, <\}$ and lower comparisons $\gtrsim \in \{\geq, >\}$, complements are $\leq^c = >$, $<^c = \geq$, $\geq^c = <$, and $>^c = \leq$. The four clause shapes, each instantiated for its strict and non-strict bound, give all eight primitive cases:

$$\mathcal{T}_{\pi}(p \hat{U}_{\lesssim d} q) = X(((x \lesssim d) \wedge \text{unch}(x) \wedge P) \vee ((x \lesssim d) \wedge Q)), \quad (5)$$

$$\mathcal{T}_{\pi}(p \hat{U}_{\gtrsim d} q) = \text{rst}(x) \wedge X((P U ((x \gtrsim d) \wedge (P U Q))) \vee (GP \wedge GFQ)), \quad (6)$$

$$\mathcal{T}_{\pi}(p \hat{R}_{\lesssim d} q) = \text{rst}(x) \wedge X(Q \vee ((x \lesssim^c d) \vee (P \wedge Q))), \quad (7)$$

$$\mathcal{T}_{\pi}(p \hat{R}_{\gtrsim d} q) = X(((x \gtrsim^c d) \wedge \text{unch}(x)) \vee ((P R Q) \vee ((x \gtrsim^c d) \wedge P))). \quad (8)$$

All clock atoms occur positively. Upper-bounded hatted Until uses strong Until, matching the mechanized clause: its witness must satisfy q while the clock remains within the upper bound. On a clock-consistent trace over divergent time, an infinite continuation cannot keep the preservation guard true forever. Lower-bounded Until retains its fairness branch, including GFQ , so that eventuality is not lost. The weak-Until clauses mark the end of bounded Release windows. Strict comparisons are represented directly; no strict bound is weakened by adjusting its constant.

The clauses have four distinct temporal roles. For upper-bounded Until, the clock is tested before any new reset: while its value remains in the allowed window, P must continue until Q appears. If the bound is passed first, no bounded witness exists and the strong-Until clause fails. For lower-bounded Until, a finite path waits for the threshold and then starts the unbounded $P U Q$ obligation. If the threshold is never reached, the second disjunct requires P at every position and Q infinitely often. Time divergence is what makes this infinite branch equivalent to a witness at distance at least d .

For upper-bounded Release, the initial Q is required and the weak-until stops either at the complementary clock comparison or when $P \wedge Q$ discharges the obligation. For lower-bounded Release, the clock remains unchanged before the threshold; after that point, the translated obligation is ordinary Release, with the endpoint alternative covering

TABLE 1
Clock comparisons used by each hatted primitive.

Primitive role	Non-strict bound	Strict bound
$\hat{U}_{\lesssim d}$ endpoint	$x \leq d$	$x < d$
$\hat{U}_{\gtrsim d}$ threshold	$x \geq d$	$x > d$
$\hat{R}_{\lesssim d}$ window end	$x > d$	$x \geq d$
$\hat{R}_{\gtrsim d}$ pre-threshold	$x < d$	$x \leq d$

TABLE 2
Finite prefix showing how a per-request reset can hide an expired older deadline. Ages are measured immediately before each event; the deadline is five time units. No continuation can repair the missed deadline at time 5.

Event i	Action	Time t_i	Oldest req.	Reset on req.
0	<i>req</i>	0	0	0
1	<i>req</i>	4	4	4
2	<i>ack</i>	6	6	2

the transition into the threshold region. Thus Until clauses use existential witnesses, whereas Release clauses maintain a universal condition over the bounded region. The hatted shift is encoded by the outer X in every clause; for restart families, the separate $\text{rst}(x)$ atom denotes the reference timestamp at the current position.

3.3 Why hatted operators and shared clocks matter

The hatted operators preserve the triggering timestamp while moving logical evaluation to the next event. The same distinction appears in the clocked encoding: the initial X moves the temporal obligation forward, while the clock retains the reference selected by its reset trace.

For instance, $\Box(e \Rightarrow \hat{\Box}_{<\theta} \neg e)$ measures the exclusion window from the triggering event. Prefixing the ordinary operator with $\hat{\Box}$ shifts the reference to the next event, while omitting $\hat{\Box}$ also constrains the triggering event itself. The three readings differ on the same timed word, as shown in Fig. 2.

Clock sharing has two layers. First, identical primitive formulas at different syntax-tree paths share one clock by construction. Second, that single clock must represent all pending activations. For example, in $\Box(\text{req} \Rightarrow \hat{\Diamond}_{\leq 5} \text{ack})$, requests at times 0 and 4 overlap. If an acknowledgment occurs at time 6, resetting at the second request makes its age appear to be two, although the first request's deadline expired at time 5. The shared-clock construction retains the older reference and does not accept this word on the basis of the newer request alone; Table 2 records both ages. It is a finite prefix: every infinite continuation still has the missed deadline for the request at time 0. More generally, for any newer request at time t_k between an older activation t_i and a witness t_j , $t_j - t_k \leq t_j - t_i$; an endpoint that satisfies the older upper-bound obligation also satisfies the newer one. The argument depends on the required P condition holding between activations and the endpoint, which is exactly what the Until residual records.

For the other primitive families, the canonical policy and residual predicates handle the direction in which restarting or preserving the clock is required (Supplement 2.2).

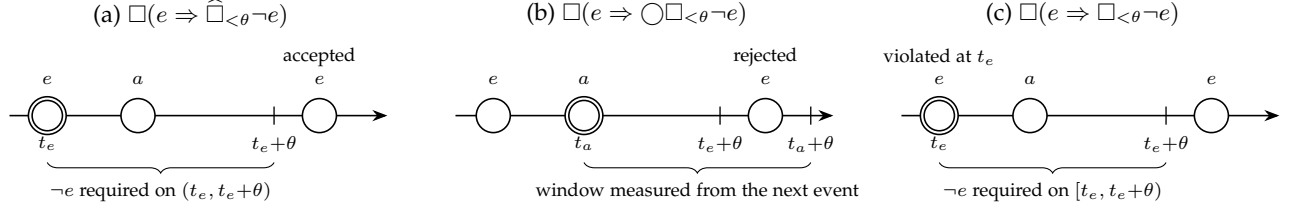


Fig. 2. The hatted sporadicity property and two ordinary-operator substitutions. A leading next operator shifts the timing reference to the following event; omitting it includes the triggering event in the window.

3.4 Canonical reset policy

The encoding quantifies over clock traces. To prove completeness with one clock for a formula that may be activated arbitrarily often, it is enough to construct one trace that gives priority to an older activation only while that activation remains useful. The proof records usefulness by residual obligations. For upper-bounded Until, the residual says that a witness is still reachable from the current clock age:

$$\begin{aligned} \text{URes}_{\lesssim}(i, v, d, p, q) &\iff \exists j \geq i. v + \text{elap}_w(i, j) \lesssim d \\ &\quad \wedge w, j \models_{\text{MTL}} q \\ &\quad \wedge \forall k \in [i, j). \\ &\quad \quad w, k \models_{\text{MTL}} p, \end{aligned}$$

where $\lesssim$ is either $\leq$ or $<$. For lower-bounded Release, the residual records that every position beyond the threshold is still covered:

$$\begin{aligned} \text{RRes}_{\geq}(i, v, d, p, q) &\iff \forall j \geq i. d \leq v + \text{elap}_w(i, j) \Rightarrow \\ &\quad \left(w, j \models_{\text{MTL}} q \right. \\ &\quad \quad \vee \exists k \in [i, j). \\ &\quad \quad \quad \left. w, k \models_{\text{MTL}} p \right), \\ \text{RRes}_{>}(i, v, d, p, q) &\iff \forall j \geq i. d < v + \text{elap}_w(i, j) \Rightarrow \\ &\quad \left(w, j \models_{\text{MTL}} q \right. \\ &\quad \quad \vee \exists k \in [i, j). \\ &\quad \quad \quad \left. w, k \models_{\text{MTL}} p \right). \end{aligned}$$

The strict predicates preserve the strict comparison itself. They are defined and shifted in the Rocq development (the full definitions and one-step lemmas are in Supplement 2.2).

The residual predicates are deliberately asymmetric. An upper-Until residual is existential: one still-reachable endpoint is enough, and that endpoint can discharge the obligation initiated at the current position. A lower-Release residual is universal: every position that has entered the threshold region must satisfy the Release condition. The canonical policy preserves a clock only when its residual remains true, so clock reuse follows from the semantic shape of the obligation rather than a syntactic rule such as “reset on every activation.” This distinction is needed for repeated activations, but the policy is not part of the generated automaton: it is a word-dependent witness used only to prove completeness.

For clock $x = F$ with current value v , the canonical policy is summarized in Table 3. “Preserve iff” means that the clock is not reset exactly under the listed condition; in the remaining cases it is reset. For the restart families, the

TABLE 3
Canonical reset policy for each timed-operator family.

Primitive clock F	Reset decision at position i
$p \hat{U}_{\leq d} q$	Preserve iff $v \leq d$, $\text{URes}_{\leq}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} q$.
$p \hat{U}_{< d} q$	Preserve iff $v < d$, $\text{URes}_{<}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} q$.
$p \hat{U}_{\geq d} q, p \hat{U}_{> d} q, p \hat{R}_{\leq d} q, p \hat{R}_{< d} q, p \hat{R}_{\geq d} q$	Reset iff $w, i \models_{\text{MTL}} F$.
$p \hat{R}_{> d} q$	Preserve iff $v < d$, $\text{RRes}_{\geq}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} p$.
	Preserve iff $v \leq d$, $\text{RRes}_{>}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} p$.

stated test is the reset condition. The canonical extension uses this decision for the formula-keyed clock and updates its value by

$$u_0 = 0, \quad u_{i+1} = \begin{cases} \Delta_i, & \text{if the policy resets at } i, \\ u_i + \Delta_i, & \text{otherwise.} \end{cases}$$

The policy depends on truth in the timed word and, through residuals, on its future. It is a semantic witness used in the completeness proof, not an algorithm executed by the translator. Its input is the timed formula itself, not a syntax-tree path, so identical occurrences share the same reset trace.

The two preservation cases retain an older reference only if the residual is still true and the current event has not discharged it. A later endpoint that satisfies the older upper-bounded Until also satisfies newer pending copies with shorter remaining prefixes. For lower-bounded Release, an old reference reaches the threshold no later than a newer one; the residual shift $(i, v) \mapsto (i + 1, v + \Delta_i)$ carries the remaining obligation. Conversely, lower-bounded Until and upper-bounded Release restart when the current formula is true; their translated continuation keeps any older obligations represented. The strict cases follow the same organization with strict residuals and guards.

3.5 Why one trace suffices for shared activations

The clock key is the primitive timed formula, not its syntax-tree path or the dynamic activation that created an obligation. Thus identical subformulas at different tree positions use the same value and reset trace; the Rocq lemma `identical_nodes_share_canonical_trace` proves this equality. Dynamic activations still impose distinct semantic obligations. The canonical policy represents them by retaining an older clock reference only while its residual obligation remains valid.

For an upper-bounded Until, let activations at $i_0 \leq i_1 < j$ share an endpoint j . The older reference gives the stronger deadline constraint,

$$t_j - t_{i_1} \leq t_j - t_{i_0} \leq d.$$

Its required p -prefix contains the suffix required by the newer activation. Consequently, preserving the older clock while $\text{URes}_{\leq}$ holds covers both activations; when the older residual fails or its endpoint has discharged it, the policy can reset for a new obligation. This is the invariant used by `complete_UhatLe` and its strict counterpart.

Lower-bounded Release has the dual universal shape. An older reference has at least the age of a newer one and therefore reaches the lower threshold no later. The predicate RRes records whether every position beyond that threshold remains covered by q or an earlier p . While the clock is preserved, the residual advances by

$$(i, v) \mapsto (i + 1, v + \Delta_i),$$

which is the one-step lemma `rge_residual_shift` used by `complete_RhatGe`. In the dual restart families, a current activation resets the clock, while the Γ branch for lower-bounded Until and weak-until continuation for upper-bounded Release retain the older obligations in the LTL formula. The same cases use strict residuals and strict guards for the strict primitives. The resulting sharing argument is about semantic obligations, not an assumption that activations are nonoverlapping.

4 FROM CLOCKED LTL TO A TBA

4.1 LTL-to-Büchi contract

The backend treats extended atoms as independent propositions. It returns a propositional Büchi automaton whose language is exactly that of the clocked-LTL formula. The theorem is stated for any such backend, deterministic or not.

Assumption 4.1 (Backend correctness). *For each formula F and each propositional word s over its atoms, $\text{PBA_accepts}(A, s)$ iff $s, 0 \models F$, where A is the backend automaton for F .*

This contract ranges over all propositional words, including assignments that do not correspond to any clock-consistent timed word. That stronger scope is used by relaxation.

4.2 Relaxation and reset completion

The clocked-LTL formula has positive occurrences of clock-related atoms, so its propositional language is upward closed in those atoms. In each backend cube, relaxation deletes negative literals over clock comparisons and reset/preservation markers and removes irrelevant clock-related atoms; contradictory cubes are discarded. Action literals are retained as transition labels. For soundness, first reconstruct an assignment satisfying the selected original cube and apply the backend contract to that word. Upward closure then transfers satisfaction to the relaxed word, which can only add truth on relevant clock-related atoms. The position-by-position reconstruction is given below.

Each clock belongs to one of two classes. A preservation clock is used with upper-bound tests and $\text{unch}(x)$, and a

restart clock is used with lower-bound tests and $\text{rst}(x)$. Reset completion converts a relaxed cube into a TBA transition as follows:

- reset a preservation clock unless the cube contains $\text{unch}(x)$;
- reset a restart clock exactly when the cube contains $\text{rst}(x)$;
- remove reset and preservation markers, retaining clock tests as the transition guard and action literals as its label.

The asymmetry is essential. Additional resets only decrease preservation clock values and hence preserve upper tests. A restart clock can be reset only when explicitly required, since decreasing it could invalidate a lower test. The full monotonicity argument is given in Supplement 1.1.

Lemma 4.2 (Reset completion). *For every accepting run of the relaxed backend automaton on a clock-consistent extension, there is a clock-consistent extension with the same timed word and initial values whose resets are exactly those prescribed by reset completion, and the same run is accepting in the resulting TBA.*

Proposition 4.3 (Exactness of relaxation and reset completion). *Let f be well formed and let A satisfy Assumption 4.1 for $\mathcal{T}(f)$. The TBA obtained from A by relaxation and reset completion accepts exactly the timed words w for which $w, 0 \models_{\text{MTL}} f$, under existential initial clock values.*

The relaxation proof uses positivity of the clock atoms in $\mathcal{T}(f)$. A run of the original Büchi automaton remains a run after relaxation. Conversely, take a run of the relaxed automaton and, at each position, reconstruct an assignment satisfying the selected original backend cube: make each extended atom false when its deleted negative literal requires this, and choose values for removed irrelevant atoms as needed by that cube. The backend contract then makes the reconstructed word a model of $\mathcal{T}(f)$. It agrees with the relaxed word on base atoms, while the relaxed word can only add truth on relevant extended atoms. Upward closure of $\mathcal{T}(f)$ therefore makes the relaxed word a model as well. Reset completion then assigns resets monotonically: preservation clocks become no larger, keeping upper guards true, and restart clocks become no smaller, keeping lower guards true. Thus the same Büchi run remains accepting. The proof is mechanized as `relax_sound`, `relax_complete`, and `complete_complete`.

Acceptance convention. The theorem quantifies existentially over the nonnegative clock values at the first event. This is a property of the accepted timed language; it does not by itself establish equivalence with an automaton whose clocks must start at zero. For example, a first transition guarded by $x \geq 1$ may be enabled for one existential initial value and disabled when the first event is at time zero. The worked example avoids dependence on that value by resetting its deadline clock on the first transition. The conventional initialization question is otherwise outside Theorem 5.3.

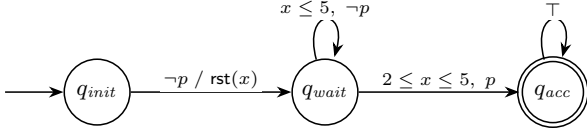


Fig. 3. A hand-simplified equivalent event-guarded TBA for f . The initial transition requires $\neg p$ and resets x . In the waiting location, every outgoing transition checks $x \leq 5$; the transition to the accepting location also requires $2 \leq x$. Thus p can first hold only at an event whose age from the origin is in $[2, 5]$.

4.3 Worked example: bounded eventuality with an exclusion window

Consider

$$f = (F_{\leq 5} p) \wedge (G_{< 2} \neg p).$$

The property requires p to be false at the temporal origin and at every later event less than two time units from it, and requires a p event by age five. Figure 3 shows a hand-simplified, equivalent one-clock event-guarded TBA. It is not the direct output of the syntax-directed translation below, which assigns separate clocks to the two distinct timed subformulas. The figure's first transition consumes the event at the origin, requires $\neg p$, and resets x . Under the event-indexed reset convention of Section 2.2, the clock value at the next event is therefore the elapsed time since the origin. The construction below follows the formula through the clocked-LTL intermediate language and the timed reinterpretation.

Step 1: The formula is in negation normal form. Expanding the derived operators and applying the timed-operator identities gives

$$\begin{aligned} f &= (\top \cup_{\leq 5} p) \wedge (\perp R_{< 2} \neg p) \\ &= (p \vee (\top \wedge \top \hat{\cup}_{\leq 5} p)) \\ &\quad \wedge (\neg p \wedge (\perp \vee \perp \hat{R}_{< 2} \neg p)). \end{aligned}$$

The current-position alternatives in the last line are the derived-operator cases of Eqs. (1) and (3).

Step 2: The two distinct primitive timed subformulas receive clocks x and y , respectively. Their clocked-LTL translation is

$$\begin{aligned} \Phi_U &= p \vee (\top \wedge X(((x \leq 5) \wedge \text{unch}(x) \wedge \top) \cup \\ &\quad ((x \leq 5) \wedge p))), \\ \Phi_R &= \neg p \wedge (\perp \vee (\text{rst}(y) \wedge \\ &\quad X(\neg p \text{ W } ((y \geq 2) \vee (\perp \wedge \neg p))))) , \\ \mathcal{T}(f) &= \Phi_U \wedge \Phi_R. \end{aligned}$$

The upper-bounded Until uses x as a preservation clock. The lower test $y \geq 2$ is the complement of the strict source bound < 2 ; the Release clause therefore permits p beginning exactly at age two.

Step 3: A Spot backend output for $\mathcal{T}(f)$ is shown on the left of Fig. 4. Its cube labels illustrate that clock guards, reset markers, preservation markers, and action atoms are independent propositions at this stage. Here $p = \text{LAct}(p)$, while $\neg p = \text{LNAct}(p)$ is a distinct proposition and $!p$ is the literal negation of $\text{LAct}(p)$. The backend may therefore satisfy a cube containing both p and $\neg p$; action consistency is

imposed after timed reinterpretation. The diagram is an illustrative output, not a state-size or performance result.

Step 4: Relaxation removes negative literals over extended clock atoms, and reset completion assigns resets according to the clock class: x is reset unless the transition carries $\text{unch}(x)$, while y is reset exactly when it carries $\text{rst}(y)$. The right side of Fig. 4 shows the resulting TBA transitions after the markers are removed. This is the boundary of the translation theorem; subsequent optimization removes the unsatisfiable action cubes and simplifies the automaton.

5 CORRECTNESS AND CLOCK BOUND

An extended word ρ adds, at every position i , a value $\text{val}_\rho(i, x) \in \mathbb{R}_{\geq 0}$ and reset decision $\text{reset}_\rho(i, x)$ for each $x \in \text{Clock}(f)$. It is clock-consistent when, for $\Delta_i = t_{i+1} - t_i$,

$$\text{val}_\rho(i+1, x) = \begin{cases} \Delta_i, & \text{reset}_\rho(i, x), \\ \text{val}_\rho(i, x) + \Delta_i, & \text{otherwise.} \end{cases} \quad (9)$$

The value at position i is the value tested by guards before the reset decision at that position. Let $\text{same_base}(\rho, w)$ mean that w is the timed-word projection of ρ .

Theorem 5.1 (Correctness of the clock encoding). *For every well-formed formula f and timed word w ,*

$$w, 0 \models_{\text{MTL}} f \iff \exists \rho. \text{same_base}(\rho, w) \wedge \text{clock_consistent}(\rho) \wedge \rho, 0 \models_{\text{LTL}} \mathcal{T}(f).$$

The existential quantifier over ρ is taken jointly over all distinct primitive timed subformulas. Soundness does not rely on choosing the canonical trace: each clock may follow any reset pattern allowed by the encoding, provided its value recurrence is respected. Completeness is the more delicate half because all activations that share a formula-keyed clock must be served simultaneously. The proof constructs the clock traces in parallel, using the residual predicates only for the preservation families and the direct restart rule for the remaining families. This result is the Rocq theorem `EncodingCorrect_proved`. Its soundness direction holds for any clock-consistent reset trace. Completeness constructs one canonical trace for all clocks together. The detailed induction, including the strict-bound arithmetic, is in Supplement 2.

Corollary 5.2 (One clock per distinct primitive timed subformula). *All occurrences of the same primitive timed subformula use the same logical clock. The encoding uses exactly the clocks in $\text{Clock}(f)$, hence at most one clock per distinct primitive timed subformula, independently of the number of syntax-tree occurrences or run-time activations.*

Proof. Every clock in $\mathcal{T}(f)$ is indexed by an element of $\text{Clock}(f)$, and `T_uses_every_clock` proves that every such element occurs in $\mathcal{T}(f)$. A single extension assigns one value and reset trace to each element of $\text{Clock}(f)$. Theorem 5.1 proves that the canonical trace suffices for every pending activation of that formula. $\square$

Define `compile_with(A)` by applying relaxation and reset completion to backend automaton A . Timed acceptance

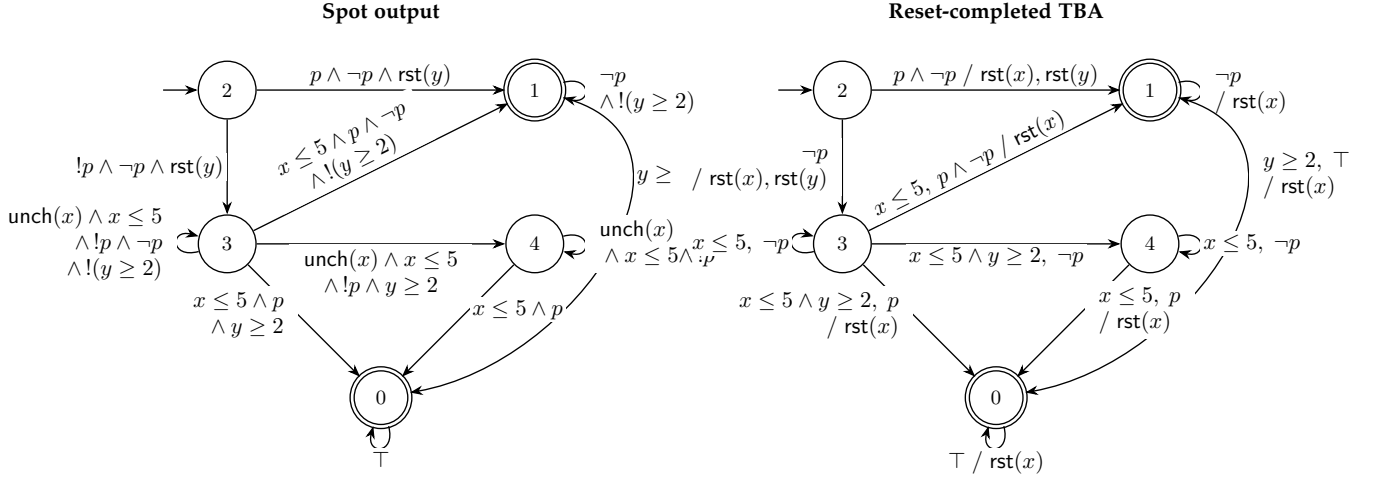


Fig. 4. Spot 2.16 output (TikZ redraw, left) and hand-derived reset-completed TBA (right; same state IDs). The raw DOT and input are in `examples/spot/`. Right-hand edges show guard, action, and resets; the markers `rst(c)` and `unch(c)` mean reset and preserve clock c . Action-atom notation is explained in the text.

is existential over clock-consistent extensions of the timed word and requires that reset decisions agree with the reset sets along an accepting run.

Theorem 5.3 (MTL-to-TBA correctness). *For every well-formed f , backend automaton A satisfying Assumption 4.1, and timed word w ,*

$$w, 0 \models_{\text{MTL}} f \iff \text{TBA_accepts}(\text{compile_with}(A), w).$$

In Rocq this is `MTL_to_TBA_correct_with`; the backend correctness is an explicit hypothesis, not a project-specific axiom.

Proof. Combine Theorem 5.1 with Proposition 4.3. The forward direction uses the canonical extension and reset-completion lemma; the reverse direction uses upward closure under relaxation, the backend contract, and clock-encoding soundness. $\square$

5.1 Semantic and arithmetic proof facts

The proof uses the standard LTL expansions of always, eventually, and weak Until. For every LTL formula A and position i ,

$$\begin{aligned} \rho, i \models_{\text{LTL}} GA &\iff \forall j \geq i, \rho, j \models_{\text{LTL}} A, \\ \rho, i \models_{\text{LTL}} FA &\iff \exists j \geq i, \rho, j \models_{\text{LTL}} A, \\ \rho, i \models_{\text{LTL}} GFA &\iff \forall n \geq i \exists j \geq n, \rho, j \models_{\text{LTL}} A, \end{aligned} \quad (10)$$

$$\begin{aligned} \rho, i \models_{\text{LTL}} A WB &\iff \left(\exists j \geq i, \rho, j \models_{\text{LTL}} B \right. \\ &\quad \wedge \forall k \in [i, j). \rho, k \models_{\text{LTL}} A \Big) \\ &\quad \vee \forall j \geq i, \rho, j \models_{\text{LTL}} A. \end{aligned} \quad (11)$$

These are the Rocq lemmas `LG_semantics`, `LF_semantics`, `LGF_semantics`, and `LW_semantics`. Time divergence also gives a future B position at any required physical distance when `GFB` holds (`GF_far`), which supplies the witness in the fairness branch of lower-bounded Until.

Clock consistency converts guards into elapsed-time bounds.

Lemma 5.4 (Accumulation under preservation). *If x is not reset in $[i, j)$, then*

$$\text{val}_\rho(j, x) = \text{val}_\rho(i, x) + \text{elap}_w(i, j).$$

Since clock values are nonnegative, $\text{elap}_w(i, j) \leq \text{val}_\rho(j, x)$. Thus an upper guard $x \leq d$ at j entails

$$\text{elap}_w(i, j) \leq d. \quad (12)$$

An upper guard $x < d$ gives the strict version.

Lemma 5.5 (Clock value after a reset). *If x is reset at i and $j \geq i + 1$, then $\text{val}_\rho(j, x) \leq \text{elap}_w(i, j)$. If it is not reset again before j , equality holds.*

A later reset only makes the clock younger. Hence a lower guard gives

$$x \geq d \implies d \leq \text{val}_\rho(j, x) \leq \text{elap}_w(i, j), \quad (13)$$

and $x > d$ entails a strict lower bound. The Rocq lemmas are `preserve_accumulate`, `after_reset_value_le_elapsed`, and `reset_preserve_value_eq_elapsed`.

5.2 Strict-bound cases

The eight hatted constructors are the primitive proof families. Ordinary timed operators have already been reduced by Eqs. (1)–(4). Strict comparisons remain strict through each translation and proof case. In particular, preservation gives $\text{elap}_w(i, j) \leq \text{val}_\rho(j, x)$, so $x < d$ implies $\text{elap}_w(i, j) < d$. After a reset, $\text{val}_\rho(j, x) \leq \text{elap}_w(i, j)$, so $x > d$ implies $\text{elap}_w(i, j) > d$.

The fairness branch of $\hat{U}_{>d}$ needs one additional step. Time divergence gives a future q at distance at least d from position $i + 1$; strict timestamp increase gives $t_{i+1} > t_i$. The same witness is therefore at distance strictly greater than d from i . This is the Rocq lemma `sound_UhatGt`.

TABLE 4
Rocq lemmas for the primitive timed proof cases. Each row denotes both strict and non-strict comparisons.

Primitive family	Soundness	Canonical completeness
$\hat{U}_{\leq d}$	sound_UhatLe	complete_UhatLe
$\hat{U}_{\geq d}$	sound_UhatGe	complete_UhatGe
$\hat{R}_{\leq d}$	sound_RhatLe	complete_RhatLe
$\hat{R}_{\geq d}$	sound_RhatGe	complete_RhatGe
$\hat{U}_{< d}$	sound_UhatLt	complete_UhatLt
$\hat{U}_{> d}$	sound_UhatGt	complete_UhatGt
$\hat{R}_{< d}$	sound_RhatLt	complete_RhatLt
$\hat{R}_{> d}$	sound_RhatGt	complete_RhatGt

5.3 Proof structure

The Rocq proof has two directions. Soundness derives the timed bounds from any clock-consistent extension; it does not assume the canonical reset policy. Completeness constructs one reset trace for all formula-keyed clocks. Its key invariant is that an older reference is retained while its residual obligation remains useful, then replaced when necessary. Structural induction handles Boolean and untimed operators, while the eight primitive timed cases are summarized in Table 4. The supplement contains the detailed case arguments, arithmetic, and theorem composition.

The local cases are assembled into the theorem `EncodingCorrect_proved`. Its completeness argument proves equivalence for every node identified by a syntax-tree path, not only for the root. This stronger induction invariant is needed because repeated formula occurrences share the same clock while the induction traverses separate paths.

What the mechanization assumes. The proof boundary is summarized in Table 5. In particular, the correctness of the untimed backend is an explicit proposition-level assumption; the timed proof does not depend on Spot or on determinism. In the included development, the theorem `MTL_to_TBA_correct_with` accepts that `contract` as an argument. The convenience theorem `MTL_to_TBA_correct` uses the core axiom `LTL_TO_BUCHI_CORRECT`; the theorem stated in this paper, `MTL_to_TBA_correct_with`, instead takes backend language equivalence as an explicit argument. The assumption audit in `proof/ASSUMPTIONS.txt` also reports the standard Rocq assumptions `ClassicalDedekindReals.sig_forall_dec`, `dependent`, `functional`, `extensionality`, `Description.constructive_definite_description`, and `Classical_Prop.classic`. These library assumptions are separate from the backend contract and from the convenience theorem’s project axiom. Parsing, backend invocation and output printing are not part of this theorem.

6 RELATED WORK

MTL and its dense-time pointwise semantics have a substantial literature, including decidability and expressiveness results [1], [2], [4]–[6]. The one-sided-bounded fragment is a useful compromise: unrestricted pointwise dense-time MTL

TABLE 5
Scope of the mechanized translation theorem.

Pipeline stage	Assurance boundary and software consequence
Formula front end	Input is a well-formed Rocq formula; parsing and finite bound encoding are outside the theorem.
Timed encoding	Rocq proves clocked-LTL equivalence using one clock per distinct primitive timed formula.
Propositional backend	Must satisfy Assumption 4.1; Spot is illustrative and unverified.
TBA construction	Rocq proves relaxation and reset completion; output has event guards and no location invariants.
Model-checker adapter	Initial clock values are existential; zero-initialization and export adapters are outside the theorem.
Tool integration	No end-to-end run; Spot regenerates only the propositional graph, and the TBA is hand-derived.

does not admit a general effective translation of all formulas to finite ordinary timed automata. Results under bounded-variability assumptions or with alternating timed automata have different scopes [9], [10]; $MTL_{0,\infty}$ retains common response and separation requirements [2], [3]. The hatted operators in this paper make the distinction between a timing reference and the current logical position explicit. They are primitive constructors in the logic and proof, so their strict pointwise behavior is not delegated to an unverified preprocessing pass.

Translations of timed logics. As Table 6 shows, Bulchev et al. and CASAAL already use timed-subformula clocks and explicit reset policies, and provide mathematical language-correctness results for their respective scopes. The overlap trace in this paper illustrates why a reset policy needs a correctness argument; it does not identify a defect in either prior tool. The incremental result here is a Rocq proof of this clocked-LTL factorization, relaxation, and reset completion, covering all eight strict and non-strict primitive cases and repeated overlapping activations. It uses real-valued bounds and an explicit backend contract, but makes no clock-count or performance comparison.

For MITL, classical constructions use temporal testers, networks of automata, or one-clock alternating automata [10]–[12]. Tools such as MightyL and MITL2Timed provide practical translations and UPPAAL-oriented workflows [13], [14]; they target a different logic and architecture. Existing techniques for merging timed obligations are related to our clock-reuse problem, but the residual predicates here are tailored to one-sided bounds and are part of the machine-checked completeness proof.

Clocked temporal logic and subformula clocks. CLTL_{Loc} extends LTL with clock comparisons and reset predicates and reduces satisfiability to a theory solver; it has also been used to encode metric logics [15]. Our clocked-LTL formula uses a similar propositional vocabulary but serves as an intermediate representation for automaton synthesis. The backend treats clock tests and reset markers as propositions; positive clock atoms make the language upward closed, while preservation markers and reset completion recover

TABLE 6

Construction-level comparison with the closest $MTL_{0,\infty}$ automata work. Exact language claims for Bulychev et al. and CASAAL are over non-Zeno infinite words. The comparison is qualitative and does not rank clock counts, performance, or proof strength.

Work	Input and output	Clocks and reset policy	Correctness and implementation evidence
Bulychev et al. [3]	Safety/co-safety fragment of $MTL_{0,\infty}$; controller-synthesis construction.	Auxiliary clocks for timed temporal subformulas; residual formulas, reset/unchanged markers, and an explicit clock-update policy.	Language-equivalence results for the stated fragments and a mathematical controller-synthesis construction.
CASAAL [7], [8]	$MTL_{0,\infty}$ to timed Büchi automata, with deterministic approximations for controller synthesis.	Clocks indexed by timed subformulas; reset/unchanged markers and a best-choice reset policy.	Exact language equivalence for the nondeterministic TBA and an implemented translation workflow.
This paper	Well-formed Rocq formulas with real bounds; exact event-guarded property TBA over infinite timed words.	One clock per distinct primitive timed formula; residual predicates and a canonical reset trace prove completeness under repeated activations.	Rocq theorem for clocked-LTL factorization and reset completion, conditional on explicit backend language equivalence; frontend and tool integration are outside the theorem.

their timed meaning. This relationship motivates the explicit relaxation proof rather than assuming that the backend understands clocks.

Event-clock logics also associate clocks with subformulas, often to record time since or until an event [9], [16], [17]. Here each distinct timed formula has one clock that may face multiple overlapping activations. As the $MTL_{0,\infty}$ constructions above illustrate, formula-indexed clocks are not themselves the novelty; this paper mechanizes the canonical reference and reset argument for its strict and non-strict primitive cases.

Recent automata-based real-time verification. Bouyer, Srivathsan, and Vishwanath improve an MTL-to-one-clock alternating timed automaton construction for finite timed words using explicit clock deactivation, identify a bounded-width fragment, and develop zone-based emptiness checking [18]. Their alternating-automaton and zone-analysis setting differs from our infinite-word TBA target, but is closely related in its treatment of clock activity. Wang et al. formally verify a transformation from finite, discrete-time MLTL formulas to regular expressions [19]; this is adjacent mechanized translation work with a different time domain and output language. TEMPORA provides pointwise MITL model checking with past and future modalities using generalized timed automata and zone-based algorithms [20]. It is an adjacent tool pipeline rather than an equivalent $MTL_{0,\infty}$ -to-TBA translation. Ho et al.’s TACAS 2026 MightyPPL translates pointwise MITL with past and Pnueli modalities into timed automata over finite and infinite timed words [21]. The paper gives mathematical construction results and proof sketches, including for reversal and tester automata, as well as a complete implementation. Its compositional tester construction sequentializes overlapping obligations and supports backends including Uppaal, TChecker, and LTSmin [21]. The QEST+FORMATS 2026 follow-up reports an implemented extension for existentially punctual MTLPP [22], [23]. In negation normal form, that input class restricts punctual subformulas to designated existential positions: the right operand of Until or Since, the left operand of Release or Trigger, or an argument

of a Pnueli modality. Following the cited definition, the QEST paper calls its subfragment without past or Pnueli modalities *one-sided MTL*; in this paper, one-sided instead describes interval shape, $[0, d]$ or $[d, \infty)$. The following containment concerns conventional source formulas using ordinary timed operators, after zero-bound reductions and restricting our bounds to integers. The mechanized AST also admits hatted timed constructors, which have no direct counterpart in the QEST grammar; the containment claim does not range over arbitrary AST terms containing those constructors. This ordinary interval syntax has no punctual subformulas and uses neither past nor Pnueli operators. The QEST class additionally permits punctual intervals in the stated positions and non-one-sided non-punctual intervals. Their response example $\Box(p \Rightarrow \Diamond_{=5} q)$ and evaluation against TEMPORA [22] illustrate the implemented extension, but do not imply support for unrestricted punctual MTL. The two papers also use different bound domains and timed-word conventions, so this syntactic relationship is not a shared-input or semantic comparison.

Our distinct claim is a Rocq-mechanized semantic theorem for the clocked-LTL factorization: one clock per distinct primitive timed subformula remains complete under arbitrarily repeated activations, including all strict and non-strict one-sided cases. This complements MightyPPL’s construction proofs and implemented pipeline; it does not replace or re-verify them. We have not compared the tools on shared inputs or measured their relative coverage, performance, or clock counts, and make no performance claim. These related-work discussions distinguish construction and assurance scope; they do not rank the tools. The Rocq development is not integrated with MightyPPL, CASAAL, or another checker.

Mechanized translations and trusted backends. Mechanized work has verified LTL-to-Büchi translations and model checkers [24]–[27]. For metric logics, Roohi and Viswanathan used PVS to identify and correct an unsound release-operator semantics in earlier MITL decision procedures. They then adapted an existing MITL-to-timed-automata construction to the corrected semantics and for-

mally proved many supporting results [28]. Their development uses continuous-time signal semantics; this paper instead verifies a translation over pointwise infinite timed words and addresses formula-keyed clocks under repeated activations. Coq has also been used for quantitative MTL monitoring [29]. That monitoring work and this theorem address different tasks: quantitative monitoring on one hand, and the dense-time pointwise translation to an MTL-to-TBA construction on the other. The theorem deliberately assumes the correctness of the LTL-to-Büchi backend. A verified LTL translator could discharge that contract if its output were adapted to the automaton interface used by the Rocq development. Our result thus isolates the timed translation proof from the backend rather than claiming that Spot itself is verified.

7 CONCLUSION

We have mechanized semantic preservation from well-formed core formulas of $\text{MTL}_{0,\infty}$ to event-guarded timed Büchi automata without location invariants and with existential initial clock valuations. The core domain has positive lower and strict upper bounds and nonnegative non-strict upper bounds; zero-lower and zero-strict-upper reductions are described, but their preprocessing is not mechanized. Under the explicit backend contract, the clocked-LTL factorization and reset completion preserve the timed language while using one clock per distinct primitive timed subformula across overlapping activations. These are the results proved by `MTL_to_TBA_correct_with`.

For tool builders, the theorem supplies a verified basis for formula-keyed clock reuse within the stated interface. The overlapping-response trace illustrates the failure mode it prevents: per-request resets can make an expired older deadline appear fresh. An executable pipeline must still discharge the front-end, backend, initialization, and export obligations summarized in Table 5; this paper reports no integrated tool run or performance comparison with CASAAL. The Rocq sources and build instructions are available at <https://github.com/efares1/coq-verified-mtl-to-tba>.

REFERENCES

- [1] R. Koymans, “Specifying real-time properties with metric temporal logic,” *Real Time Syst.*, vol. 2, no. 4, pp. 255–299, 1990. [Online]. Available: <https://doi.org/10.1007/BF01995674>
- [2] R. Alur, T. Feder, and T. A. Henzinger, “The benefits of relaxing punctuality,” *Journal of the ACM (JACM)*, vol. 43, no. 1, pp. 116–146, 1996.
- [3] P. Bulychyev, A. David, K. Larsen, and G. Li, “Efficient controller synthesis for a fragment of $\text{MTL}(0,\infty)$,” *Acta Informatica*, vol. 51, no. 3–4, pp. 165–192, Jan. 2014.
- [4] D. D’Souza and P. Prabhakar, “On the expressiveness of MTL in the pointwise and continuous semantics,” *International Journal on Software Tools for Technology Transfer*, vol. 9, no. 1, pp. 1–4, 2007.
- [5] J. Ouaknine and J. Worrell, “On the decidability of metric temporal logic,” in *20th Annual IEEE Symposium on Logic in Computer Science (LICS 2005)*. IEEE Computer Society, 2005, pp. 188–197.
- [6] P. Bouyer, F. Chevalier, and N. Markey, “On the expressiveness of TPTL and MTL,” in *Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2005)*, ser. Lecture Notes in Computer Science, vol. 3821. Springer, 2005, pp. 432–443.
- [7] G. Li, P. G. Jensen, K. G. Larsen, A. Legay, and D. B. Poulsen, “Practical controller synthesis for $\text{MTL}_{0,\infty}$,” in *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software*, ser. SPIN 2017. New York, NY, USA: Association for Computing Machinery, 2017, pp. 102–111. [Online]. Available: <https://doi.org/10.1145/3092282.3092303>
- [8] G. Li, “CASAAL: A tool for translating $\text{MTL}_{0,\infty}$ into timed Büchi automata,” Online, 2017, available at the CASAAL tool page. [Online]. Available: <https://lcs.ios.ac.cn/~ligy/tools/CASAAL/>
- [9] D. Ničković and N. Piterman, “From MTL to deterministic timed automata,” in *Formal Modeling and Analysis of Timed Systems (FORMATS 2010)*, ser. Lecture Notes in Computer Science, vol. 6246. Springer, 2010, pp. 152–167.
- [10] T. Brihaye, M. Estiévenart, and G. Geeraerts, “On MITL and alternating timed automata over infinite words,” 2014. [Online]. Available: <https://arxiv.org/abs/1406.4395>
- [11] O. Maler, D. Nickovic, and A. Pnueli, “From MITL to timed automata,” in *Formal Modeling and Analysis of Timed Systems (FORMATS 2006)*, ser. Lecture Notes in Computer Science, vol. 4202. Springer, 2006, pp. 274–289.
- [12] T. Ferrère, O. Maler, D. Ničković, and A. Pnueli, “From real-time logic to timed automata,” *Journal of the ACM*, vol. 66, no. 3, 2019, art. no. 19.
- [13] T. Brihaye, G. Geeraerts, H.-M. Ho, and B. Monmege, “MightyL: A compositional translation from MITL to timed automata,” in *Computer Aided Verification*, R. Majumdar and V. Kunčák, Eds. Cham: Springer International Publishing, 2017, pp. 421–440.
- [14] “MITL2Timed: Modifying Oddoux and Gastin LTL2BA to timed automata generation,” GitHub repository. [Online]. Available: <https://github.com/yzh89/MITL2Timed>
- [15] M. M. Bersani, M. Rossi, and P. San Pietro, “A tool for deciding the satisfiability of continuous-time metric temporal logic,” *Acta Informatica*, vol. 53, no. 2, pp. 171–206, 2016.
- [16] R. Alur, L. Fix, and T. A. Henzinger, “Event-clock automata: A determinizable class of timed automata,” *Theoretical Computer Science*, vol. 211, no. 1–2, pp. 253–273, 1999.
- [17] J.-F. Raskin and P.-Y. Schobbens, “The logic of event clocks: Decidability, complexity and expressiveness,” *Journal of Automata, Languages and Combinatorics*, vol. 4, no. 3, pp. 247–286, 1999.
- [18] P. Bouyer, B. Srivathsan, and V. Vishwanath, “Model-checking real-time systems: Revisiting the alternating automaton route,” in *Foundations of Software Science and Computation Structures (FoSSaCS 2025)*, ser. Lecture Notes in Computer Science, vol. 15691. Springer Nature Switzerland, 2025, pp. 399–421.
- [19] Z. Wang, K. Kosaian, and K. Y. Rozier, “Formally verifying a transformation from MLTL formulas to regular expressions,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2025)*, ser. Lecture Notes in Computer Science, vol. 15696. Springer, 2025, pp. 254–275.
- [20] S. Akshay, P. Contractor, P. Gastin, R. Govind, and B. Srivathsan, “TEMPORA: Efficient verification of metric temporal properties with past in pointwise semantics,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, ser. Lecture Notes in Computer Science, vol. 16505. Springer Nature Switzerland, 2026, pp. 640–659.
- [21] H.-M. Ho, S. N. Krishna, K. Madnani, R. Majumdar, and P. Pandya, “MightyPPL: Model checking MITL with past and Pnueli modalities,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, ser. Lecture Notes in Computer Science, vol. 16505. Springer Nature Switzerland, 2026, pp. 457–479.
- [22] —, “MightyPPL: Towards model checking MTL,” in *Quantitative Evaluation of Systems and Formal Modeling and Analysis of Timed Systems (QEST+FORMATS 2026)*, ser. Lecture Notes in Computer Science, vol. 16913. Springer, 2027, pp. 216–225.
- [23] —, “MightyPPL: Towards model checking MTL,” 2026. [Online]. Available: <https://arxiv.org/abs/2609.19073>
- [24] A. Schimpf, S. Merz, and J.-G. Smaus, “Construction of Büchi automata for LTL model checking verified in Isabelle/HOL,” in *Theorem Proving in Higher Order Logics (TPHOLs 2009)*, ser. Lecture Notes in Computer Science, vol. 5674. Springer, 2009, pp. 424–439.
- [25] J. Esparza, P. Lammich, R. Neumann, T. Nipkow, A. Schimpf, and J.-G. Smaus, “A fully verified executable LTL model checker,” in *Computer Aided Verification (CAV 2013)*, ser. Lecture Notes in Computer Science, vol. 8044. Springer, 2013, pp. 463–478.
- [26] A. Schimpf and P. Lammich, “Converting linear-time temporal logic to generalized Büchi automata,” *Archive of Formal*

Proofs, 2014, https://isa-afp.org/entries/LTL_to_GBA.html, formal proof development.

- [27] B. Seidl and S. Sickert, “A compositional and unified translation of LTL into ω -automata,” *Archive of Formal Proofs*, https://isa-afp.org/entries/LTL_Master_Theorem.html, 2019.
- [28] N. Roohi and M. Viswanathan, “Revisiting MITL to fix decision procedures,” in *Verification, Model Checking, and Abstract Interpretation (VMCAI 2018)*, ser. Lecture Notes in Computer Science, vol. 10747. Springer, 2018, pp. 474–494.
- [29] A. Chattopadhyay and K. Mamouras, “A verified online monitor for metric temporal logic with quantitative semantics,” in *Runtime Verification (RV 2020)*, ser. Lecture Notes in Computer Science, vol. 12399. Springer, 2020, pp. 383–403.